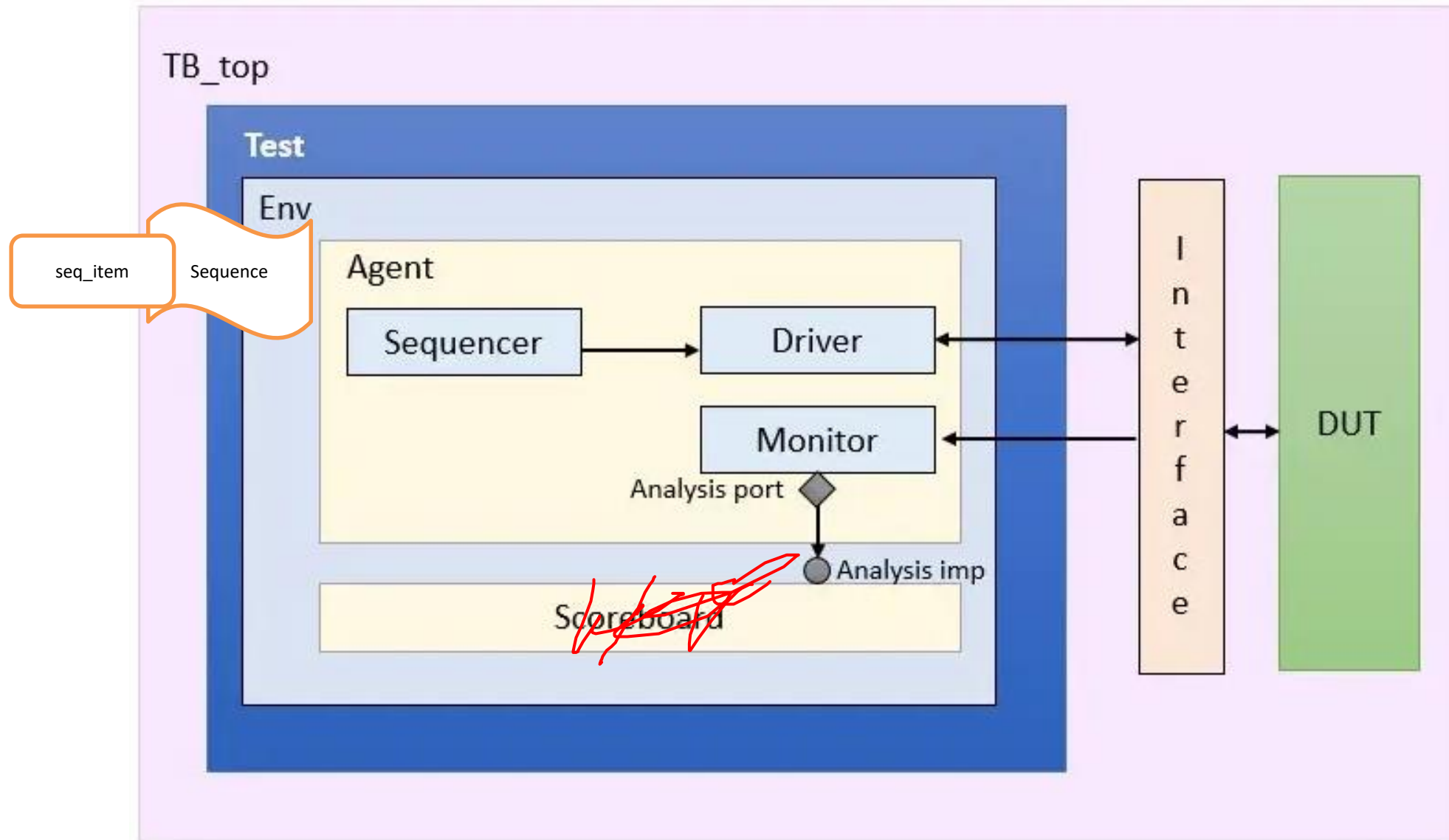


UVM Phasing and Synchronization + UVM Factory

Yasir Farooq

Recall the UVM based Testbench



UVM_Extract_Phase

- **UVM_Extract_Phase is executed in Bottom_Up Fashion**
- **UVM_Extract_Phase is used**
 - Extract any remaining data and final state information from scoreboard and testbench components
 - Probe the DUT (via zero-time hierarchical references and/or backdoor accesses) for final state information.
 - Compute statistics and summaries.
 - Display final state information
 - Close files.

UVM_Report_Phase

- UVM_Report_Phase is executed in Bottom_Up Fashion
- UVM_Report_Phase is used
 - Report test results.
 - Write results to file.

UVM_Final_Phase

- UVM_Final_Phase is executed in Bottom_Up Fashion
- UVM_Final_Phase is used
 - Close files.
 - Terminate co-simulation engines.

Run-Time Phases

Run-Time Phases	Description
uvm_pre_reset_phase	Before reset is asserted.
uvm_reset_phase	Reset is asserted.
uvm_post_reset_phase	After reset is de-asserted.
uvm_configure_phase	Runtime initial configurations for the DUT.

Run-Time Phases

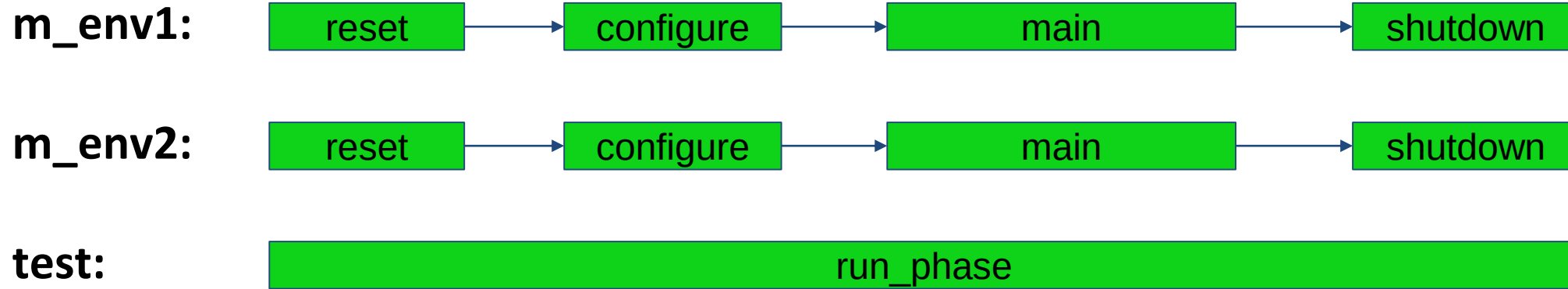
Run-Time Phases	Description
uvm_pre_main_phase	Before the primary test stimulus starts.
uvm_main_phase	Primary test stimulus.
uvm_post_main_phase	After enough of the primary test stimulus.
uvm_pre_shutdown_phase	Before things settle down.
uvm_shutdown_phase	Letting things settle down
uvm_post_shutdown_phase	After things have settled down.

Synchronization

```
class env extends uvm_env;
...
    task reset_phase (uvm_phase phase);
    ...
    task configure_phase (uvm_phase phase);
    ...
    task main_phase (uvm_phase phase);
    ...
    task shutdown_phase (uvm_phase phase);
    ...
endclass
```

```
class test extends uvm_test;
...
    env m_env1;
    env m_env2;
    function void build_phase(uvm_phase phase);
        m_env1 = env :: type_Id ::
create("m_env1",this);
        m_env2 = env :: type_Id ::
create("m_env2",this);
        ...
    task run_phase (uvm_phase phase);
    ...
endclass
```

Synchronization

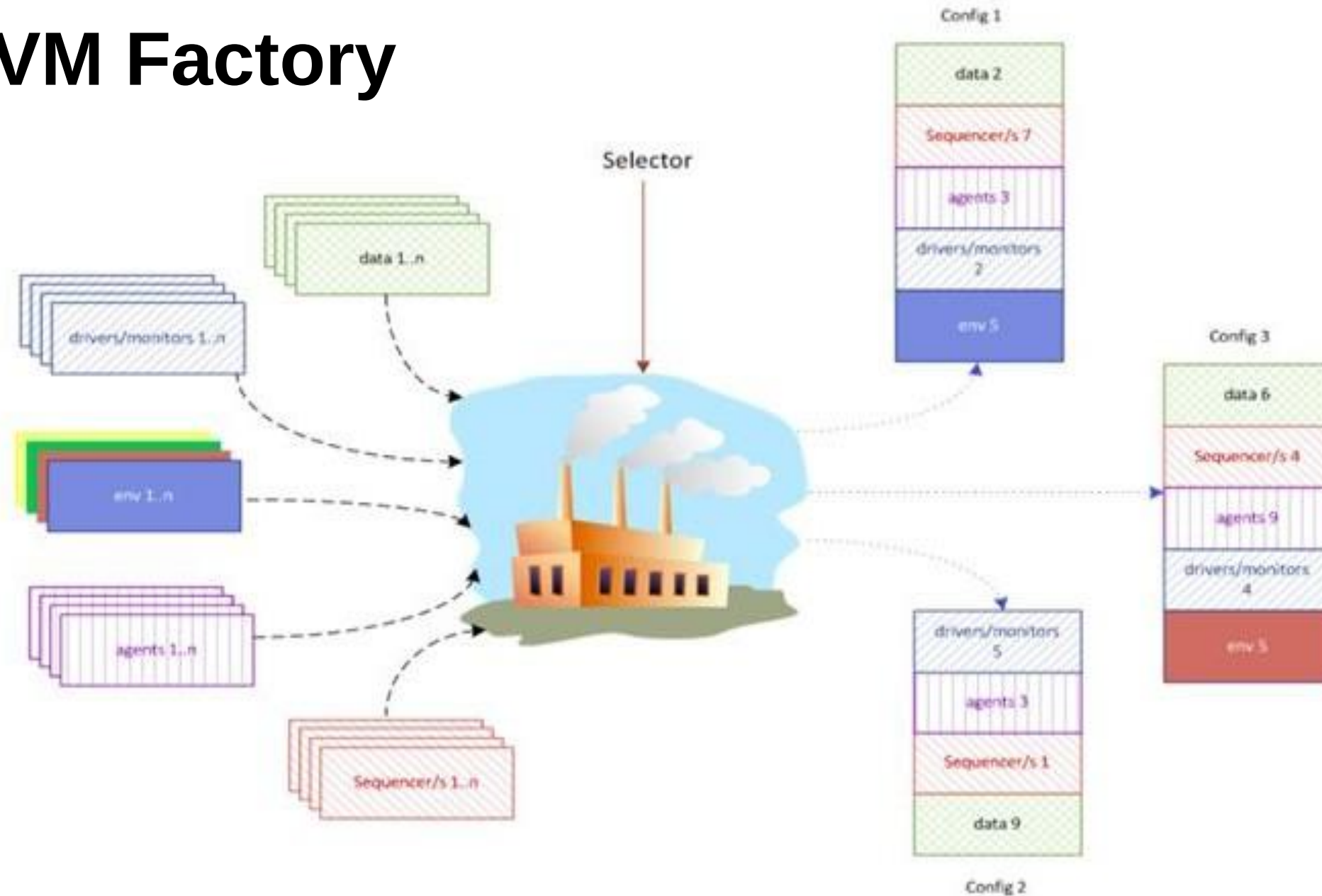


Dynamic Construction using UVM Factory

Dynamic Construction

- **uvm_factory**
 - An associated array of classes, indexed by string. It is a database that has the knowledge of all user defined classes which are extended from UVM base classes and is used as a lookup table for dynamically creating testbench components
 - Use to instantiate/create uvm objects and components i.e., **type_id::create** method
 - Replaces the new() function of system verilog to create class objects
 - Only one instance of factory is present in the given simulation termed as singleton

UVM Factory



UVM Factory

Advantages of Factory:

- Dynamic(run-time) creation of objects and components
- Overriding objects at run-time (by instance or by type)
- We can configure the component or object created from factory inside the component scope or even outside its scope
(i.e., we can set the type of the object using override methods in uvm environment which will be created down the hierarchy in uvm agent)

UVM Factory

- UVM factory allows substituting the base class objects by any of its inherited child class objects
- Using the factory involves three basic operations:
 - Registering the object and component types with the factory
 - Construction of class object using the static method create
 - Configuring the factory with type and instance overrides

Registering Class with Factory

UVM utility macros are used for registering classes with uvm factory

- **`uvm_component_utils** : for static uvm components
 - ``uvm_component_utils(class_name)`
- **`uvm_object_utils** : for dynamic uvm components
 - ``uvm_object_utils(class_name)`

These macros contain code that will register the given type with the factory

UVM Component Construction

UVM component instantiation is done using the create() method which is a static method of type_id class

type_name::type_id::create(string name, uvm_component parent)

Object Factory Use: *create*

```
class yapp_packet extends uvm_sequence_item;  
  rand bit [5:0] length;  
  rand bit [7:0] payload[];  
  `uvm_object_utils_begin(yapp_packet)  
  ...  
  constraint length_const { length > 0; length < 64; }  
endclass
```

Adds yapp_packet to factory

```
class yapp_tx_monitor extends uvm_monitor;  
  yapp_packet packet;  
  
  task collect_packet();  
    packet = new("packet");  
    packet = yapp_packet::type_id::create("packet", this);  
    ...  
  endtask  
  ...  
endclass
```

Instead of new...

...use create instead

Handle type

Static create call

Instance name
and parent pointer

- Almost all instances should be made by calling factory `create` instead of constructor `new`

Factory Component Overriding

Factory override methods are used to override the base class objects with child class objects

- **Type Overrides**
 - `set_type_override_by_type`
 - `set_type_override_by_name`
- **Instance Override**
 - `set_inst_override_by_type`
 - `set_inst_override_by_name`

Type Versus Instance Overrides

Set type overrides replaces ALL instances:

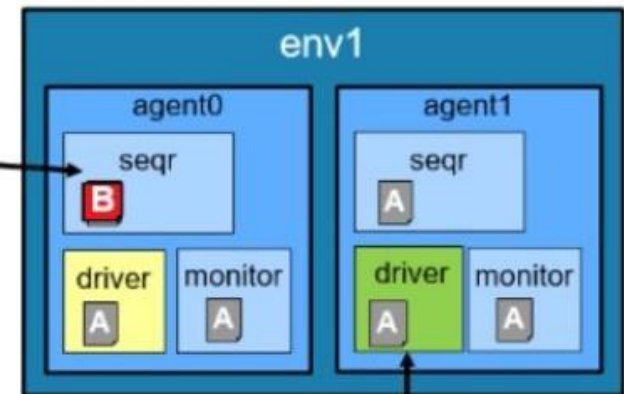
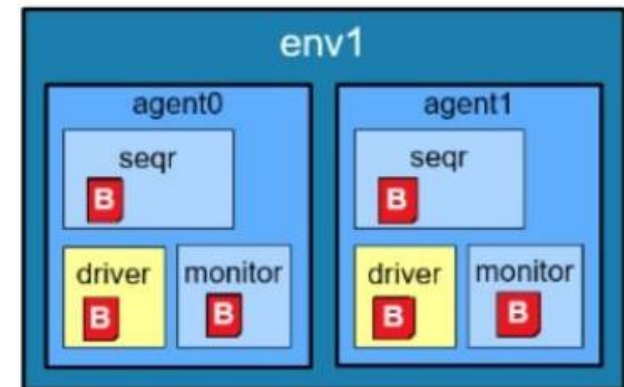
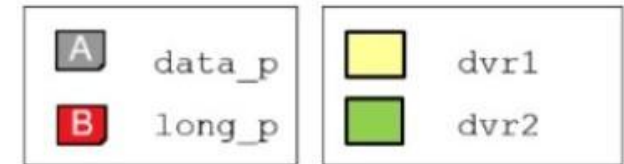
- For a specific test, replace all `data_p` packets with `long_p` packets

```
set_type_override_by_type(data_p::get_type(),  
                           long_p::get_type());
```

Set instance overrides replace specific instances:

- Replace `data_p` instances in `agent0` sequencer
- Replace `agent1` driver `dvr1` with `dvr2`

```
set_inst_override_by_type("env1.agent0.seqr",  
                           data_p::get_type(), long_p::get_type());  
set_inst_override_by_type("env1.agent1.driver",  
                           dvr1::get_type(), dvr2::get_type());
```



Lab Exercise

Lab 4 Understanding Dynamic Construction using UVM Factory and Component Overriding

To learn how to dynamically construct UVM components using the UVM Factory and override them using type and instance overrides.